

Lab 05

Lakhvinder Singh

15th April 2026

Lab 05: Making static maps

Read the instructions COMPLETELY before starting the lab

This lab builds directly on the in-class frankenmap exercise and your readings from Lovelace, Ch. 9 (<https://r.geocompx.org/adv-map.html>). You should have the in-class notebook available for reference.

Formatting your submission

This lab must be placed into a public repository on GitHub (www.github.com). Before the due date, submit **on Canvas** a link to the repository. I will then download your repositories and run your code. The code must be contained in either a .R script or a .Rmd markdown document. As I need to run your code, any data you use in the lab must be referenced using **relative path names**. Finally, answers to questions I pose in this document must also be in the repository at the time you submit your link to Canvas. They can be in a separate text file, or if you decide to use an RMarkdown document, you can answer them directly in the doc.

Introduction

In class we built the “frankenmap” as a group exercise — three teams working on pieces that were assembled into a single visualization. This lab asks you to build that same map on your own, but in a more deliberate way: **piece by piece, verifying each component before moving to the next**.

The key insight from the in-class exercise is that **tmap** is designed around composable pieces. Every map is just a base shape plus layers plus aesthetics:

```
map <- tm_shape(data) + tm_polygons() + # base
      tm_shape(data2) + tm_lines() +   # next layer
      tm_compass() + tm_scalebar()     # layout elements
```

And maps stored as variables can be added together:

```
map_full <- map_part_1 + map_part_2 + map_part_3
```

This lab walks through each component of the frankenmap systematically. By the end of Task 4, you will have a complete inset map that integrates all of them. Task 5 asks you to produce a second, entirely original map.

All data are in `../data/static_mapping/` relative to this file’s location.

```
library(sf)
```

```
## Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE
```

```
library(terra)
```

```
## terra 1.9.1
```

```

library(tmap)
library(tidyverse)

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.0      v readr      2.1.6
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.1      v tibble    3.3.1
## v lubridate  1.9.4      v tidyr     1.3.2
## v purrr      1.2.1

## -- Conflicts ----- tidyverse_conflicts() --
## x tidyr::extract() masks terra::extract()
## x dplyr::filter()  masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

library(grid) # needed for inset maps in Task 4 - part of base R, no install required

##
## Attaching package: 'grid'
##
## The following object is masked from 'package:terra':
##
##     depth

```

Task 1: The state scale — Ohio choropleth

Reference: Lovelace Ch 9.2.3–9.2.4 (color settings, choropleth mapping)

In the frankenmap, Group 1 was responsible for a state-scale choropleth of Ohio counties. You will build that component here.

1.1 — Load and join

Load the Ohio county geometry from `oh_counties.gpkg`. Also load the 2020 Census demographic data from `oh_counties_DP2020.csv`.

```

counties <- st_read("./data/static_mapping/oh_counties.gpkg")

## Reading layer `oh_counties' from data source
##   `/Users/lakhvindarsingh/GitHub/OLD_Environmental_data_analysis_in_R/Environmental_data_analysis_in_
##   using driver `GPKG'
## Simple feature collection with 88 features and 18 fields
## Geometry type: MULTIPOLYGON
## Dimension:      XY
## Bounding box:   xmin: -84.8203 ymin: 38.40342 xmax: -80.5187 ymax: 42.32713
## Geodetic CRS:   NAD83

demo_data <- read_csv("./data/static_mapping/oh_counties_DP2020.csv")

## Rows: 89 Columns: 57
## -- Column specification -----
## Delimiter: ","
## chr  (2): geoid, name
## dbl (55): poptotal, male, mu5, m5to9, m10to14, m15to19, m20to24, m25to29, m3...
##

```

```
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Before joining, inspect the GEOID column in **both** objects. You should notice something about the format of the GEOID in the CSV that requires attention before a join will work. Resolve this discrepancy, then perform a `left_join()`.

Hint: Try `head()` on both GEOIDs and compare carefully.

```
# Read data
counties <- st_read("./data/static_mapping/oh_counties.gpkg")
```

```
## Reading layer `oh_counties' from data source
##   `/Users/lakhvindarsingh/GitHub/OLD_Environmental_data_analysis_in_R/Environmental_data_analysis_in_
##   using driver `GPKG'
## Simple feature collection with 88 features and 18 fields
## Geometry type: MULTIPOLYGON
## Dimension:      XY
## Bounding box:   xmin: -84.8203 ymin: 38.40342 xmax: -80.5187 ymax: 42.32713
## Geodetic CRS:   NAD83
```

```
census <- read_csv("./data/static_mapping/oh_counties_DP2020.csv")
```

```
## Rows: 89 Columns: 57
## -- Column specification -----
## Delimiter: ","
## chr (2): geoid, name
## dbl (55): poptotal, male, mu5, m5to9, m10to14, m15to19, m20to24, m25to29, m3...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# Inspecting GEOIDs
head(counties$GEOID)
```

```
## [1] "39063" "39003" "39085" "39047" "39017" "39115"
```

```
head(census$geoid)
```

```
## [1] "0400000US39" "0500000US39001" "0500000US39003" "0500000US39005"
## [5] "0500000US39007" "0500000US39009"
```

```
# Fix census GEOID
census <- census %>%
  mutate(
    GEOID = stringr::str_sub(geoid, -5)
  )
```

```
# Joining datasets
counties_joined <- counties %>%
  left_join(census, by = "GEOID")
```

```
# Verify the join worked
nrow(counties_joined)
```

```
## [1] 88
```

```
nrow(counties)
```

```
## [1] 88
```

```
head(counties_joined$GEOID)
```

```
## [1] "39063" "39003" "39085" "39047" "39017" "39115"
```

Verify the join worked: how many rows does your joined object have? Does it match what you'd expect?

1.2 — Choose and justify a variable

Choose **one** demographic variable from the Census data. If it is a raw count, normalize it appropriately (e.g., as a proportion of total population). Add a brief comment in your code explaining what the variable represents and why you find it interesting.

```
counties_joined <- counties_joined %>%
```

```
  mutate(
    pop_65plus = m65to69 + m70to74 + m75to79 + m80to84 + mover84 +
                  f65to69 + f70to74 + f75to79 + f80to84 + fover84,
    prop_65plus = pop_65plus / poptotal
  )
```

```
# The variable which i chosed is proportion of elderly individuals (age 65+) in each county, calculated
```

1.3 — Build the Ohio-scale map

Using your joined data, build a choropleth map of all Ohio counties. Requirements:

- Color the counties by your chosen variable using an **explicit** classification scheme — do not leave it at the default. Choose from "equal", "jenks", "quantile", or "pretty" and set your number of classes.
- Style the borders with non-default line properties (adjust at least one of: color, width, or line type)
- Add a scale bar
- Give your legend a meaningful, human-readable title (not just the raw variable name)
- Store the finished map in a variable called `map_ohio`

```
map_ohio <- tm_shape(counties_joined) +
  tm_polygons(
    fill = "prop_65plus",
    style = "quantile",
    n = 5,
    palette = "YlOrRd",
    fill.legend = tm_legend(title = "Proportion Age 65+")
  ) +
  tm_borders(
    col = "black",
    lwd = 1.5
  ) +
  tm_scalebar(position = c("right", "bottom")) +
  tm_layout(
    main.title = "Proportion of Population Aged 65+ in Ohio Counties",
    legend.outside = TRUE,
    frame = FALSE
  )
```

```
##
```

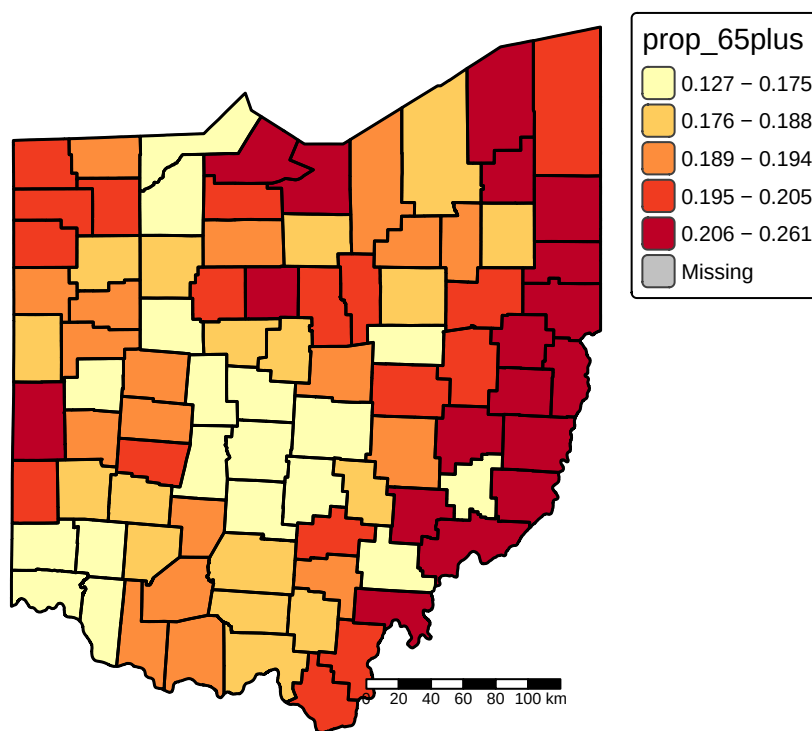
```
## -- tmap v3 code detected -----
```

```
## [v3->v4] `tm_polygons()`: instead of `style = "quantile"`, use fill.scale =
## `tm_scale_intervals()`.
```

```
## i Migrate the argument(s) 'style', 'n', 'palette' (rename to 'values') to
## 'tm_scale_intervals(<HERE>)'
## [v3->v4] `tm_layout()`: use `tm_title()` instead of `tm_layout(main.title = )`
map_ohio

## [cols4all] color palettes: use palettes from the R package cols4all. Run
## `cols4all::c4a_gui()` to explore them. The old palette name "YlOrRd" is named
## "brewer.yl_or_rd"
## Multiple palettes called "yl_or_rd" found: "brewer.yl_or_rd", "matplotlib.yl_or_rd". The first one,
##
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
```

Proportion of Population Aged 65+ in Ohio Counties



Reflection question 1 (answer in your repo): Justify your choice of classification method. How would the map's message change if you used a different method? Reference specific class boundaries in your answer.

I used quantile classification because it places an equal number of counties in each class, making it easier to compare relative differences in the proportion of people aged 65+ across Ohio. For example, the highest class (around ~0.206–0.261) clearly highlights counties with relatively older populations.

If I used an equal interval method, the ranges would be evenly spaced, but many counties would likely fall into middle classes, reducing visual contrast. Using Jenks would group counties based on natural clusters, but class sizes would be uneven, making comparisons less consistent.

Task 2: The local scale — Portage and Summit counties

Reference: Lovelace Ch 9.2.2–9.2.3 (map objects, aesthetics)

Group 2 was responsible for the detail view: municipal boundaries, parks, and water features for the Portage and Summit county study area. You will build this component here, layer by layer.

2.1 — Prepare the study area

Filter the county geometry to Portage and Summit counties. Store the result as `sa`. You will also need a bounding box polygon of the study area for use in the inset map later — create it using `sf::st_bbox()` and `sf::st_as_sfc()` and store it as `oh_region`.

```
# Filtering to Portage and Summit counties
sa <- counties %>%
  dplyr::filter(NAME %in% c("Portage", "Summit"))

# Create bounding box polygon for inset
oh_region <- sf::st_as_sfc(sf::st_bbox(sa))
```

2.2 — Municipal boundaries with labels

Load `oh_places.gpkg`. Before doing any spatial operations, check the CRS against `sa` and reproject if necessary. Spatially filter to the study area using `sf::st_intersection()`.

Build a map layer showing municipal boundaries with text labels for place names. In `tmap`, labels can be added using `tm_text()` as a separate layer, or via the `labels` argument in some geometry functions.

```
# Load places
places <- sf::st_read("./data/static_mapping/oh_places.gpkg")

## Reading layer `oh_places' from data source
##   `/Users/lakhvindarsingh/GitHub/OLD_Environmental_data_analysis_in_R/Environmental_data_analysis_in_
##   using driver `GPKG'
## Simple feature collection with 1215 features and 16 fields
## Geometry type: MULTIPOLYGON
## Dimension:      XY
## Bounding box:   xmin: -84.8202 ymin: 38.40342 xmax: -80.51879 ymax: 41.97752
## Geodetic CRS:   NAD83

# Match CRS
if (sf::st_crs(places) != sf::st_crs(sa)) {
  places <- sf::st_transform(places, sf::st_crs(sa))
}

places_sa <- sf::st_intersection(places, sa) %>%
  sf::st_make_valid()

## Warning: attribute variables are assumed to be spatially constant throughout
## all geometries

# Creating label points (centroids)
places_pts <- sf::st_centroid(places_sa)

## Warning: st_centroid assumes attributes are constant over geometries

# Map
tm_shape(places_sa) +
  tm_borders(col = "blue", lwd = 1) +
tm_shape(places_pts) +
  tm_text("NAME", size = 0.5)
```



```
## Warning: attribute variables are assumed to be spatially constant throughout
## all geometries
```

```
# Rename column to fix legend title
parks_sa <- parks_sa %>%
  dplyr::rename(`Park Type` = FEATTYPE)
```

```
# Map
tm_shape(sa) +
  tm_borders(col = "grey40", lwd = 1) +
tm_shape(parks_sa) +
  tm_polygons(
    fill = "Park Type",
    palette = "Greens"
  ) +
  tm_borders(col = "darkgreen", lwd = 0.5) +
  tm_layout(
    main.title = "Parks by Type in Portage and Summit Counties"
  )
```

```
##
```

```
## -- tmap v3 code detected -----
```

```
## [v3->v4] `tm_tm_polygons()`: migrate the argument(s) related to the scale of
## the visual variable `fill` namely 'palette' (rename to 'values') to fill.scale
## = tm_scale(<HERE>).
```

```
## [v3->v4] `tm_layout()`: use `tm_title()` instead of `tm_layout(main.title = )`
```

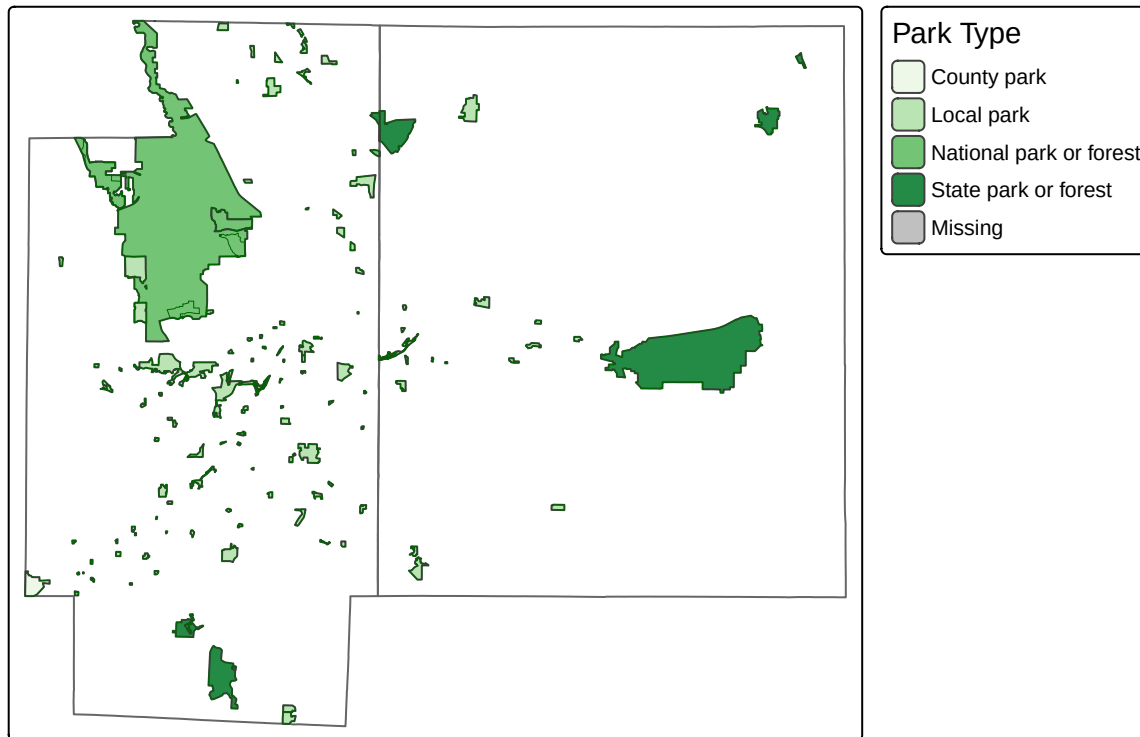
```
## [cols4all] color palettes: use palettes from the R package cols4all. Run
```

```
## `cols4all::c4a_gui()` to explore them. The old palette name "Greens" is named
```

```
## "brewer.greens"
```

```
## Multiple palettes called "greens" found: "brewer.greens", "matplotlib.greens". The first one, "brewer.greens", is used.
```


Parks by Type in Portage and Summit Counties



2.4 — Water features and park intersections

The linear water data for the two counties are stored in separate directories:

- `../data/static_mapping/tl_2022_39133_linearwater/` (Portage County)
- `../data/static_mapping/tl_2023_39153_linearwater/` (Summit County)

Load both shapefiles, combine them using `dplyr::bind_rows()`, and reproject to match `sa`.

Next, determine which stream segments **spatially intersect** a park polygon. Use `sf::st_intersects()` to identify these segments and add a new column to your water features data frame (e.g., `intersects_park`) with a value of `TRUE` or `FALSE`. Use this column to symbolize the streams differently.

```
# Loading water data
```

```
water_portage <- sf::st_read("../data/static_mapping/tl_2022_39133_linearwater/")
```

```
## Reading layer `tl_2022_39133_linearwater' from data source
```

```
##   `/Users/lakhvindarsingh/GitHub/OLD_Environmental_data_analysis_in_R/Environmental_data_analysis_in
```

```
##   using driver `ESRI Shapefile'
```

```
## Simple feature collection with 276 features and 5 fields
```

```
## Geometry type: LINESTRING
```

```
## Dimension:      XY
```

```
## Bounding box:   xmin: -81.39332 ymin: 40.98823 xmax: -81.00231 ymax: 41.34776
```

```
## Geodetic CRS:   NAD83
```

```
water_summit  <- sf::st_read("../data/static_mapping/tl_2023_39153_linearwater/")
```

```
## Reading layer `tl_2023_39153_linearwater' from data source
```

```
##   `/Users/lakhvindarsingh/GitHub/OLD_Environmental_data_analysis_in_R/Environmental_data_analysis_in
```

```
##   using driver `ESRI Shapefile'
```

```
## Simple feature collection with 388 features and 5 fields
```

```

## Geometry type: LINESTRING
## Dimension:      XY
## Bounding box:  xmin: -81.68825 ymin: 40.90666 xmax: -81.39163 ymax: 41.35116
## Geodetic CRS:  NAD83

# Combining datasets
water <- dplyr::bind_rows(water_portage, water_summit)

# Match CRS
if (sf::st_crs(water) != sf::st_crs(sa)) {
  water <- sf::st_transform(water, sf::st_crs(sa))
}

# Spatial filtering to study area
water_sa <- sf::st_intersection(water, sa) %>%
  sf::st_make_valid()

## Warning: attribute variables are assumed to be spatially constant throughout
## all geometries

# Determining which streams intersect parks
intersections <- sf::st_intersects(water_sa, parks_sa)

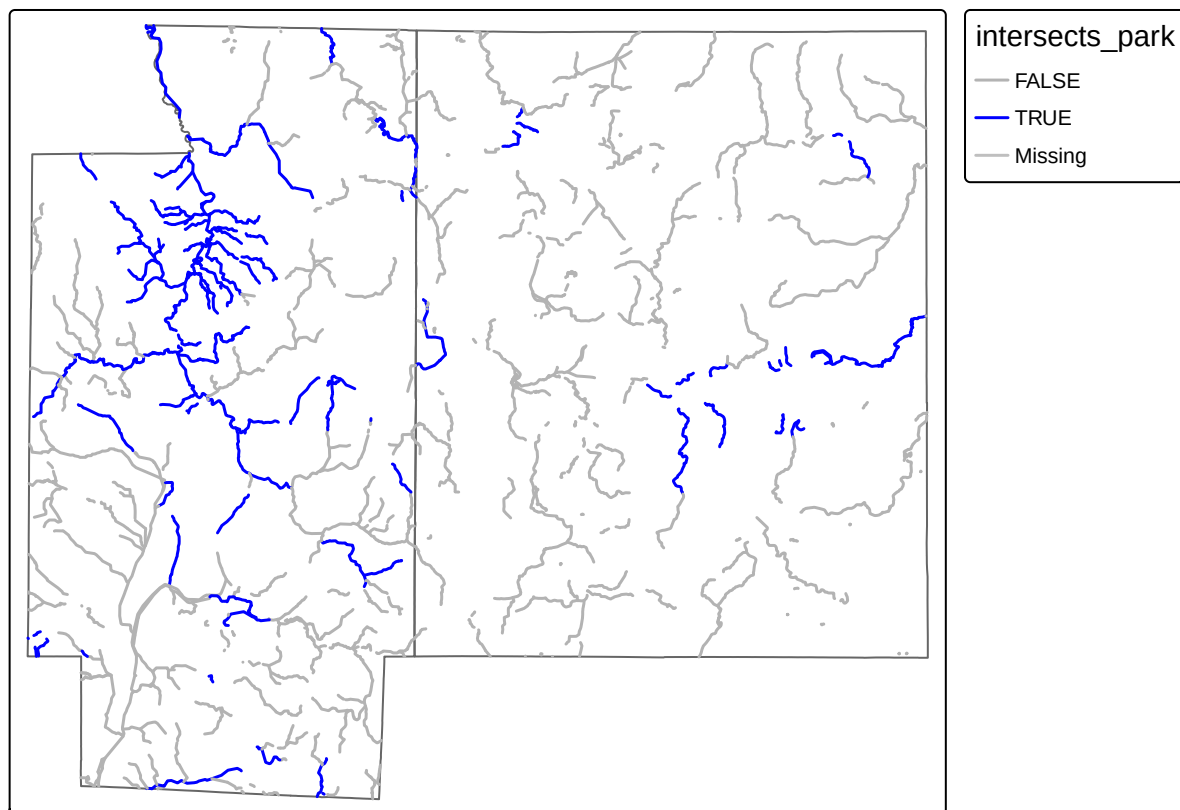
water_sa <- water_sa %>%
  mutate(intersects_park = lengths(intersections) > 0)

# Map streams symbolized by intersection
tm_shape(sa) +
  tm_borders(col = "grey40") +
  tm_shape(water_sa) +
  tm_lines(
    col = "intersects_park",
    palette = c("grey70", "blue"),
    lwd = 1.5,
    col.legend = tm_legend(title = "Intersects Park")
  )

##

## -- tmap v3 code detected -----
## [v3->v4] `tm_tm_lines()`: migrate the argument(s) related to the scale of the
## visual variable `col` namely 'palette' (rename to 'values') to col.scale =
## tm_scale(<HERE>).

```



2.5 — Assemble the local map

Combine your places, parks, and water layers into a single map variable called `map_local`.

```
tm_shape(sa) +
  tm_borders(col = "black", lwd = 1.2) +

# Parks layer (polygons)
tm_shape(parks_sa) +
  tm_polygons(
    fill = "Park Type",
    palette = "Greens",
    colorNA = "transparent"
  ) +
  tm_borders(col = "darkgreen", lwd = 0.5) +

# Water layer (lines)
tm_shape(water_sa) +
  tm_lines(
    col = "intersects_park",
    palette = c("FALSE" = "grey70", "TRUE" = "blue"),
    lwd = 1.2,
    col.legend = tm_legend(title = "Stream intersects park")
  ) +

# Municipal boundaries
tm_shape(places_sa) +
  tm_borders(col = "blue", lwd = 0.8) +
```


Reflection question 2 (answer in your repo): Describe the cartographic decisions you made about visual hierarchy in this map. Which features should visually dominate, and what specific symbol choices (size, color, transparency) did you make to achieve that? Why?

I designed the map so that parks are the most visually prominent feature since they are the main focus. I used filled polygons in shades of green to make them stand out clearly, while darker outlines help define their boundaries. Streams are shown as a secondary layer, with blue lines highlighting those that intersect parks and light grey lines for others, so important ecological connections are easy to see without overwhelming the map. Municipal boundaries are included with thin blue lines to provide context, but they are kept subtle so they don't compete with the parks or streams. I used color contrast, line thickness, and layering to guide the viewer's attention from parks to key streams, while keeping background information visible but not distracting.

Task 3: Raster integration and layout elements

Reference: Lovelace Ch 9.2.5 (layouts and map elements)

Group 3 was responsible for integrating the DEM, adding layout elements, and combining everything. Let's start with the raster component.

3.1 — Prepare the DEM

Load `neoh_dem.tif` using `terra::rast()`.

Before cropping, **project** it to match the CRS of `sa` using `terra::project()`. Then crop to the study area extent. Add a comment explaining your workflow choices.

```
# Load DEM raster
dem <- terra::rast("./data/static_mapping/neoh_dem.tif")
```

```
# Projecting DEM to match study area CRS
dem_proj <- terra::project(dem, sf::st_crs(sa)$wkt)
```

```
## |-----|-----|-----|-----|=====
```

```
# Cropping DEM to study area extent
dem_crop <- terra::crop(dem_proj, terra::vect(sa))
```

```
# The DEM is first projected to match the CRS of the study area to ensure proper spatial alignment with
```

Tip: Projecting a large raster can be slow. Cropping AFTER projecting (as shown here) crops the projected version; cropping BEFORE projecting (to a smaller area first) can be faster. There are trade-offs — consider what's appropriate here.

3.2 — Build the integrated local map

Now build a more complete version of the local map that incorporates the DEM. Requirements:

- The DEM as the base layer, using a continuous color scale appropriate for elevation (e.g., `YlGn`, `terrain`, or another sequential palette)
- The Portage and Summit county polygons overlaid with **semi-transparency** (`fill_alpha`) so the elevation shows through
- Your parks and water features from Task 2
- A north arrow: `tm_compass()`
- A scale bar

Store this as `map_local_elev`.

```
map_local_elev <-
  tm_shape(dem_crop) +
  tm_raster(
    col.scale = tm_scale_continuous(
      values = "-YlGn",
      limits = c(800, 1300)
    ),
    col.legend = tm_legend(title = "Elevation")
  ) +

  # Study area
  tm_shape(sa) +
  tm_polygons(
    fill = "grey",
    fill_alpha = 0.2,
    col = "black",
    lwd = 1
  ) +

  # Parks
  tm_shape(parks_sa) +
  tm_polygons(
    fill = "Park Type",
    palette = "Greens",
    fill_alpha = 0.9
  ) +
  tm_borders(col = "darkgreen", lwd = 0.5) +

  # Streams
  tm_shape(water_sa) +
  tm_lines(
    col = "intersects_park",
    palette = c("FALSE" = "grey85", "TRUE" = "blue"),
    lwd = 1
  ) +

  # Municipal boundaries
  tm_shape(places_sa) +
  tm_borders(col = "grey80", lwd = 0.5) +

  # Labels
  tm_shape(places_pts) +
  tm_text("NAME", size = 0.35, remove.overlap = TRUE) +

  # Map elements
  tm_scalebar(position = c("right", "bottom")) +
  tm_compass(position = c("left", "top"), size = 1.2) +
  tm_layout(
    main.title = "Elevation, Parks, and Streams in Portage and Summit Counties",
    legend.outside = TRUE,
    frame = FALSE
  )
```

```
##

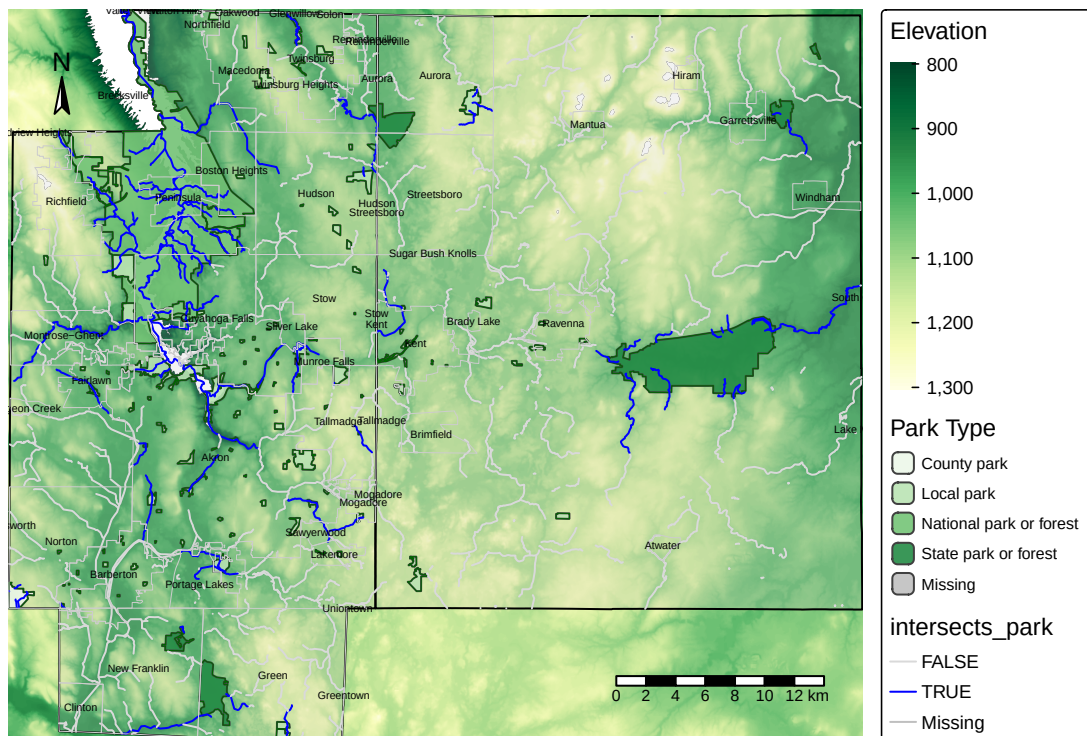
## -- tmap v3 code detected -----

## [v3->v4] `tm_tm_polygons()` : migrate the argument(s) related to the scale of
## the visual variable `fill` namely 'palette' (rename to 'values') to fill.scale
## = tm_scale(<HERE>).
## [v3->v4] `tm_tm_lines()` : migrate the argument(s) related to the scale of the
## visual variable `col` namely 'palette' (rename to 'values') to col.scale =
## tm_scale(<HERE>).
## [v3->v4] `tm_layout()` : use `tm_title()` instead of `tm_layout(main.title = )`

map_local_elev

## Values have been found that are lower than the lowest limit. These 'outliers' have been set to NA. U
## Values have been found that are higher than the upper limit. These 'outliers' have been set to NA. U
## [cols4all] color palettes: use palettes from the R package cols4all. Run
## `cols4all::c4a_gui()` to explore them. The old palette name "-YlGn" is named
## "yl_gn" (in long format "brewer.yl_gn") [cols4all] color palettes: use palettes from the R package co
## `cols4all::c4a_gui()` to explore them. The old palette name "Greens" is named
## "brewer.greens" Multiple palettes called "greens" found: "brewer.greens", "matplotlib.greens". The fi
## [plot mode] legend/component: Some components or legends are too "high" and are
## therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
```

Elevation, Parks, and Streams in Portage and Summit Counties



Reflection question 3 (answer in your repo): How does adding the elevation layer change the map's visual story? Did it create any visual hierarchy problems, and if so, how did you address them?

Adding the elevation layer helps show how the landscape influences where parks and streams are located, making the map more informative. However, the DEM initially stood out too much and made it harder to see other features. I fixed this by using a softer color palette and adjusting transparency so parks remain the main focus, streams are secondary, and elevation stays in the background as supporting context.

Task 4: Multi-panel arrangement and inset maps

Reference: Lovelace Ch 9.2.6 (faceted maps) and Ch 9.2.7 (inset maps)

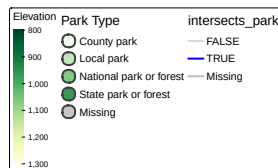
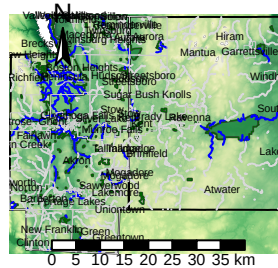
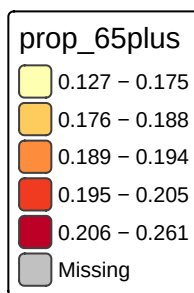
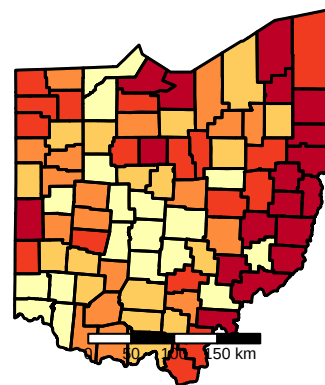
This task has two parts. The first uses `tmap_arrange()` — a straightforward approach to multi-panel maps. The second builds a proper inset, which is more powerful but more involved.

4.1 — Side-by-side with `tmap_arrange()`

Use `tmap_arrange()` to display `map_ohio` and `map_local_elev` side by side. Experiment with the `ncol` and `asp` arguments to get a reasonable layout.

```
tmap_arrange(
  map_ohio + tm_layout(legend.outside = TRUE),
  map_local_elev + tm_layout(legend.outside = TRUE),
  ncol = 2,
  widths = c(1, 1),
  asp = 0.8
)
```

```
## [cols4all] color palettes: use palettes from the R package cols4all. Run
## `cols4all::c4a_gui()` to explore them. The old palette name "YlOrRd" is named
## "brewer.yl_or_rd"
## Multiple palettes called "yl_or_rd" found: "brewer.yl_or_rd", "matplotlib.yl_or_rd". The first one,
##
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
## Values have been found that are lower than the lowest limit. These 'outliers' have been set to NA. U
##
## Values have been found that are higher than the upper limit. These 'outliers' have been set to NA. U
##
## [cols4all] color palettes: use palettes from the R package cols4all. Run
## `cols4all::c4a_gui()` to explore them. The old palette name "-YlGn" is named
## "yl_gn" (in long format "brewer.yl_gn")
## [cols4all] color palettes: use palettes from the R package cols4all. Run
## `cols4all::c4a_gui()` to explore them. The old palette name "Greens" is named
## "brewer.greens"
##Multiple palettes called "greens" found: "brewer.greens", "matplotlib.greens". The first one, "brewer
##
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
```

Reflection question 4 (answer in your repo): `tmap_arrange()` is easy to use, but it has real limitations compared to a true inset. What are at least two limitations? (Think about how the two maps relate spatially and what information is or isn't communicated.)

In my side-by-side layout, the local map appears disconnected, making it difficult to understand where it is located within Ohio. It also cannot embed one map inside another, which limits how effectively scale and location can be communicated. In contrast, the inset map directly links the two scales by placing the local map within the Ohio map, making the spatial context much clearer.

4.2 — Building the inset map

A true inset map places a small context map *inside* the main map — a common technique for showing where a study area sits within a larger geographic context. Achieving this in `tmap` requires the `grid` package's viewport system.

Below is a partially complete code skeleton. **Read every comment carefully** before filling in the blanks. The concepts behind the viewports are more important than getting the aesthetics perfect on the first try.

```
# ---- Step 1: Build the inset (context) map ----
# This is the small map of all of Ohio that shows where our study area is.
# oh_region is the bounding box polygon you created in Task 2.1.

map_inset <- tm_shape(counties_joined) + tm_polygons() + # fill in: all Ohio counties
  tm_shape(oh_region) + tm_borders(lwd = 2, col = "red") + # fill in: border width
  tm_layout(bg.color = "lightblue") +
  tm_scalebar(position = c("left", "bottom"))

# ---- Step 2: Normalize dimensions ----
# A viewport is defined in "normalized parent coordinates" (npc) from 0 to 1.
```

```

# If we just use width and height directly, maps with very different aspect ratios
# will appear distorted. norm_dim() calculates the width and height as fractions
# of the longest dimension, so that the aspect ratio is preserved.

norm_dim <- function(obj) {
  bbox <- st_bbox(obj)
  width  <- bbox[["xmax"]] - bbox[["xmin"]]
  height <- bbox[["ymax"]] - bbox[["ymin"]]
  w <- width / max(width, height)
  h <- height / max(width, height)
  return(unit(c(w, h), "npc"))
}

# Calculate dimensions for both maps
# The main map is based on the study area bounding box (oh_region)
# The inset map is based on all of Ohio
main_dim <- norm_dim(oh_region) # fill in: use oh_region
ins_dim  <- norm_dim(counties_joined) # fill in: use the all-Ohio sf object

# ---- Step 3: Define viewport positions ----
# A viewport is a rectangular region of a graphics device.
# main_vp defines the size of the main map within the graphics window.
# ins_vp defines the size AND position of the inset.

main_vp <- viewport(width = main_dim[1], height = main_dim[2])

ins_vp <- viewport(
  width  = ins_dim[1] * 0.33,          # scale the inset to 40% of normalized Ohio width
  height = ins_dim[2] * 0.33,          # scale proportionally
  x      = unit(0.08, "npc") - unit(0.5, "cm"), # right edge of device minus a small margin
  y      = unit(0.10, "npc"),          # near the bottom of the device
  just   = c("right", "bottom")        # anchor point for the inset
)

# ---- Step 4: Draw ----
# grid.newpage() clears the device.
# print(..., vp = ...) draws a tmap within a specific viewport.
# pushViewport() makes the main viewport the "active" coordinate system,
#   so the inset is positioned relative to the main map, not the whole device.

grid.newpage()
print(map_local_elev, vp = main_vp)

```

```

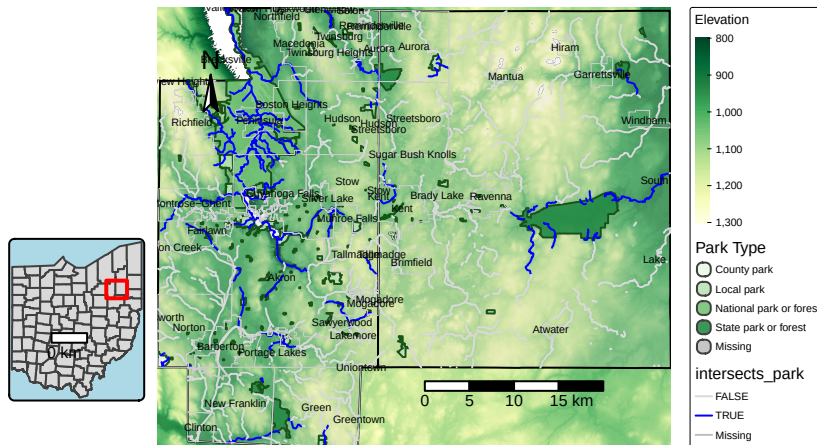
## Values have been found that are lower than the lowest limit. These 'outliers' have been set to NA. U
## Values have been found that are higher than the upper limit. These 'outliers' have been set to NA. U
## [cols4all] color palettes: use palettes from the R package cols4all. Run
## `cols4all::c4a_gui()` to explore them. The old palette name "-YlGn" is named
## "yl_gn" (in long format "brewer.yl_gn")
## [cols4all] color palettes: use palettes from the R package cols4all. Run
## `cols4all::c4a_gui()` to explore them. The old palette name "Greens" is named
## "brewer.greens"

```

```
## Multiple palettes called "greens" found: "brewer.greens", "matplotlib.greens". The first one, "brewer.greens",
##
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
```

```
pushViewport(main_vp)
print(map_inset, vp = ins_vp)
```

Elevation, Parks, and Streams in Portage and Summit Counties



Your first attempt will not look perfect — that is expected and intentional. You will need to:

- Fill in the four blanks (___)
- Adjust the scale factor (0.4) and margin (0.5, "cm") to improve the inset's size and placement
- Add a title using `tm_title()` or `tm_layout(title = ...)` to `map_local_elev`

Note: Getting the position right requires iterating. Try different values for the scale factor (e.g., 0.3, 0.5) and the margin. A scale factor around 0.35–0.45 and bottom-right or bottom-left placement usually works well for this study area.

Reflection question 5 (answer in your repo): In your own words, explain what `norm_dim()` is doing and why it is necessary. What specific visual problem would occur if you skipped that step and passed the raw coordinate extents to `viewport()` directly?

`norm_dim()` basically rescales the map's width and height so it keeps the correct shape when placed in a viewport. It makes sure the aspect ratio is preserved. If you skip it and use raw extents, the map would look stretched or squished because the viewport would force it into the wrong proportions.

Task 5: Your own static map

Using whatever spatial data, extent, and domain you choose, create an original static map. Requirements:

1. It must include **vector AND raster data** used together in a meaningful way
2. It must include spatial data representing a **social process** (e.g., political boundaries, land use, population) AND an **environmental process** (e.g., elevation, water, vegetation, climate)
3. The map should **stand on its own** and communicate its purpose without additional explanatory text — if a reader needs to ask “what am I looking at?”, the map hasn't done its job
4. Good coding practices: comments, relative paths, clean variable names

You may use data from any of the course datasets (CBW, Ohio, Lake Erie) or data you obtain independently. If using external data, document the source in a comment.

```
# Load data
nlcd      <- rast("./data/ohio/neoh_nlcd.tif")
oh_counties <- read_sf("./data/static_mapping/oh_counties.gpkg")
water_portage <- read_sf("./data/static_mapping/tl_2022_39133_linearwater/tl_2022_39133_linearwater.shp")
water_summit <- read_sf("./data/static_mapping/tl_2023_39153_linearwater/tl_2023_39153_linearwater.shp")

# Filter, reproject, crop, mask
sa      <- oh_counties |> filter(NAME %in% c("Portage", "Summit"))
sa_albers <- st_transform(sa, crs(nlcd))
nlcd_sa  <- crop(nlcd, vect(sa_albers)) |> mask(vect(sa_albers))

# Reclassify: 1=Urban, 2=Natural, 3=Agricultural
rcl <- matrix(c(21,24,1, 41,43,2, 90,95,2, 81,82,3), ncol=3, byrow=TRUE)
nlcd_rc <- classify(nlcd_sa, rcl, others = NA)
levels(nlcd_rc) <- data.frame(value=1:3,
                              label=c("Urban / Developed",
                                       "Natural (Forest & Wetland)",
                                       "Agricultural"))

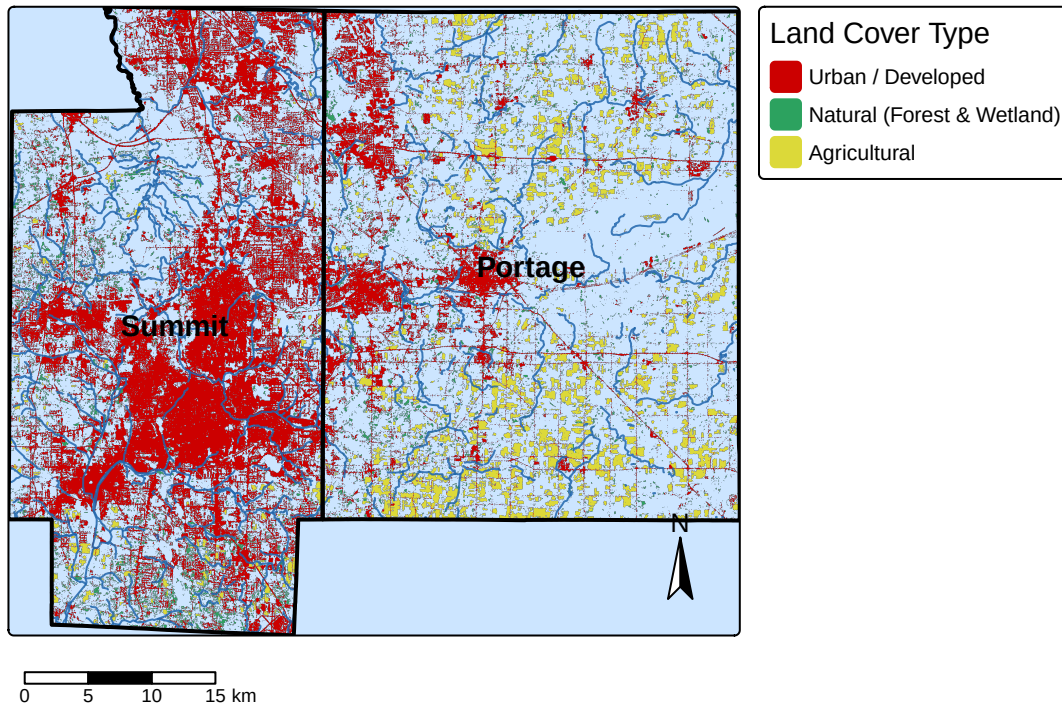
nlcd_rc <- project(nlcd_rc, "EPSG:4269", method="near")
sa_nad <- st_transform(sa, 4269)
water_nad <- bind_rows(st_transform(water_portage, 4269),
                      st_transform(water_summit, 4269))
nlcd_rc <- crop(nlcd_rc, vect(sa_nad)) |> mask(vect(sa_nad))

# Colors
lc_cols <- c("Urban / Developed"      = "#cc0000",
             "Natural (Forest & Wetland)" = "#2ca25f",
             "Agricultural"          = "#dcd939")

# Map
tm_shape(nlcd_rc) +
  tm_raster(col.scale = tm_scale_categorical(values = lc_cols),
           col.legend = tm_legend(title = "Land Cover Type")) +
tm_shape(water_nad) +
  tm_lines(col = "#2166ac", lwd = 1.0, col_alpha = 0.85) +
tm_shape(sa_nad) +
  tm_borders(col = "black", lwd = 2) +
  tm_text("NAME", size = 0.9, fontface = "bold") +
tm_title("Urban, Natural & Agricultural Land in NE Ohio") +
tm_scalebar(position = tm_pos_out("center", "bottom"), text.size = 0.6) +
tm_compass(position = c("right", "bottom"), size = 2) +
tm_layout(legend.outside = TRUE, legend.outside.position = "right",
          frame = TRUE, bg.color = "#cce5ff")
```

```
## [plot mode] fit legend/component: Some legend items or map components do not
## fit well, and are therefore rescaled.
## i Set the tmap option `component.autoscale = FALSE` to disable rescaling.
```

Urban, Natural & Agricultural Land in NE Ohio



Questions

1. Describe and justify your cartographic choices in Task 1 (Ohio choropleth). Specifically address: your classification method, your color palette, and how these choices serve the map's intended communication goal. Would the map tell a different story with different choices?

I used a quantile classification because it spreads the data evenly across classes, which makes it easier to compare counties visually. Since my data were not perfectly evenly distributed, this helped avoid having too many counties in one category. I chose a sequential color palette (light to dark) so that higher values stand out clearly and the map is intuitive to read. These choices support the goal of showing relative differences across Ohio. If I had used a different classification, like equal intervals, or a different color scheme, the map could look more uniform or highlight different areas, so the interpretation would definitely change.

2. In Task 2.4 you identified stream segments that intersect parks. Describe your approach step by step, as if explaining it to a classmate who hasn't done spatial data analysis before. Why is this distinction cartographically meaningful (*i.e., why might a map reader care which streams run through parks)?

To do this, I first loaded the park boundaries and stream layers and made sure they were in the same coordinate system. Then I used `st_intersection()` to find where the streams overlap with park areas. This function basically keeps only the portions of streams that fall inside park polygons. This is cartographically meaningful because it highlights where natural water systems exist within protected areas. A map reader might care about this because these streams could be important for conservation, recreation, or management decisions, and they may be better protected than streams outside parks.

3. In Task 4, you used both `tmap_arrange()` and the `grid` viewport approach to create multi-panel maps. What are the trade-offs between these two methods? In what professional or analytical context would you prefer one over the other?

`tmap_arrange()` is much simpler and quicker for placing maps side by side. It works well when the layout is straightforward. The `grid` viewport approach, on the other hand, gives more control over positioning and allows for more customized layouts, like inset maps. The trade-off is that it takes more effort and is less intuitive. I would use `tmap_arrange()` for quick comparisons or exploratory work, and the `grid` approach

when I need a more polished or customized figure, such as for a report or publication.

4. Compare the cartographic decisions you made individually in this lab to the decisions your group made collectively during the in-class frankenmap exercise. Where did your choices diverge, and what explains the difference?

Since I wasn't present for the in-class frankenmap activity, I can't directly compare the exact design choices made by the group. However, I expect that the group map likely included more layers and different design ideas since multiple people contributed. In contrast, my individual map focused on clarity and a single main message, which made it more straightforward to interpret. Group work often brings more creativity and perspectives, but it can also make the map more complex and harder to read if not carefully coordinated.

5. Describe your original map in Task 5: what spatial story are you trying to tell, did you achieve your communication goal, and what would you change given more time?

I designed this map to show where land is urban, agricultural, or natural in Portage and Summit counties. By grouping land cover into a few categories, I made the urban-rural contrast easy to see. I think I mostly achieved my goal. I can clearly see Summit County's dense urban core around Akron, while Portage appears more mixed with agriculture and scattered development. The stream network adds another layer by showing how water moves through both developed and natural areas, so the map is easy to understand without much explanation. If I had more time, I would add parks to highlight protected areas, and include more counties to better show the full urban-rural gradient. I would also add a small inset map so viewers can understand where this area is within Ohio.